

Documentazione Test

1. Test di Unità

Package: utility

Classe Testata: Keyboard

- **Test 1 (testReadIntConInputNonValido):**

- **Cosa testa:** Il comportamento difensivo di `readInt()` in caso di input non numerico.
- **Descrizione:** Inserendo la stringa "ciao" come input, il test verifica che il metodo restituisca esattamente `Integer.MIN_VALUE` (-2147483648) e che il contatore degli errori venga incrementato di 1. Questo conferma che l'eccezione `NumberFormatException` viene catturata internamente senza propagarsi al chiamante.

- **Test 2 (testReadDoubleConInputNonValido):**

- **Cosa testa:** Il comportamento difensivo di `readDouble()` in caso di input non numerico.
- **Descrizione:** Inserendo la stringa "ciao" come input, il test verifica che il metodo restituisca `Double.NaN` e che il contatore degli errori venga incrementato, gestendo l'eccezione internamente senza interrompere il flusso del programma.

- **Test 3 (testReadCharConInputNonValido):**

- **Cosa testa:** Il comportamento di `readChar()` in caso di errore di lettura.
- **Descrizione:** Verifica che, fornendo un input che causa un'eccezione, il metodo restituisca `Character.MIN_VALUE` e incrementi il contatore errori, mantenendo la coerenza con gli altri metodi della classe.

- **Test 4 (testIncrementoContatoreErroriMultiplo):**

- **Cosa testa:** Il corretto accumulo del contatore errori attraverso errori consecutivi.
- **Descrizione:** Eseguendo due letture errate consecutive (prima `readInt()` con "errore1\n" poi `readDouble()` con "errore2\n"), il test verifica che il contatore venga incrementato correttamente a 2, dimostrando che lo stato viene mantenuto tra diverse chiamate.

- **Test 5 (testReadIntConInputValido):**

- **Cosa testa:** Il corretto funzionamento di `readInt()` con input valido.
- **Descrizione:** Inserendo il valore "42\n", il test verifica che il metodo restituisca il valore intero 42 e che il contatore degli errori rimanga a 0, confermando che il comportamento normale non è influenzato dalla logica di gestione degli errori.

- **Test 6 (testReadWordSingoloToken):**

- **Cosa testa:** La corretta estrazione di un singolo token tramite `readWord()` .
- **Descrizione:** Con input "Hello World\n", il test verifica che il metodo utilizzi `StringTokenizer` per estrarre solo il primo token ("Hello") senza incrementare il contatore degli errori.

- **Test 7 (testReadStringConcatenaToken):**

- **Cosa testa:** La concatenazione di tutti i token fino a fine riga tramite `readString()` .
- **Descrizione:** Con input "Hello World Test\n", verifica che il metodo recuperi il primo token e poi iteri concatenando tutti i token successivi fino a `endOfLine()` , restituendo "Hello World Test" e preservando gli spazi originali come previsto dall'implementazione, senza incrementare gli errori.

- **Test 8 (testReadCharConBuffer):**

- **Cosa testa:** Il meccanismo di buffering in `readChar()` che salva i caratteri rimanenti in `current_token` .
- **Descrizione:** Con input "ab\ncd\n", dimostra che la prima lettura estrae 'a' e salva "b" nel buffer, la seconda legge 'b' senza consumare nuovo input, e la terza consuma la riga successiva leggendo 'c', tutto mantenendo gli errori a 0.